

Overview of Statistical Learning and Modeling – 36-600

Lab 1R: R Boot Camp
Due Thursday, August 28, 11:59pm

Trisha Nair

tinytex::install_tinytex()

If you're reading this: congratulations! It means that you have successfully installed `R` and `RStudio` on your computer and that you are ready to dive into building the basic skills that will lead you to being able to manipulate data frames: tables of data where each row represents an observation (e.g., a person) and each column represents a measurement (e.g., the height or weight of that person).

Before we start, note that when you insert code into each "code chunk" below (the sections delimited with `"{r}"` and `"}"`), you can test it in a number of ways:

- at the upper-right corner of a code chunk, there is a green triangle – clicking on that runs all the code in the chunk
- also at the upper-right corner of a code chunk, there is a white triangle pointing downward toward a green line – clicking that symbol runs all the code in previous chunks up to and **not** including the current one (attached to which you pressed the button; this means if you hit this button on the very first code chunk of a document, **nothing** will execute)
- you can highlight any amount of code (and only code) and, on Windows, press Control+Enter, or on a Mac, press Command+Return, and the highlighted code will execute
- you can click on "Knit," which will run *all code in the file* and output an html file with the results

Question 1

Each column of a data frame is itself a vector. So let's start with some basic vector manipulation. Use the `c()` function to define a vector `x` with four values: 1, 2, 1, and 8. (*You should replace the "FILL ME IN" statement below with your answer.*) Note that vectors are homogeneous (all of the same type), with the most common types being `double` (or `numeric`), `character`, and `logical`. The vector you define has type `double`. Check this by typing `typeof(x)` and noting the output. Then type `x[1]`. What do you see? (Indicate your answer by, on a new line in the code chunk, typing one number symbol # [which denotes a comment] and then afterwards typing your answer in words [and/or numbers].)

```
x <- c(1, 2, 1, 8)
x[1]

## [1] 1
```

If the value(s) inside the square bracket is/are numeric, then that/those elements of the vector are displayed. (Note: `R` counts from 1, not 0.) If the value(s) are logical, then only those elements with value `TRUE` are displayed. This will make more sense below.

Question 2

Now define a vector `y` with four values: 2, 2, 5, 8. Then add `x` and `y`, and multiply `x` and `y`. Note that the following operators are using to carry out basic math in `R`:

Operation	Description
+	addition
-	subtraction
*	multiplication
/	division
^	exponentiation
%%	modulus (i.e., remainder)
%/%	division with (floored) integer round-off

```
y <- c(2,2,5,8)
print(x+y)

## [1] 3 4 6 16

print(x * y)

## [1] 2 4 5 64
```

What you should observe are vectors with four numbers each. Note that `R` did not require you to loop over the vector indices, i.e., `R` did not make you add `x[1]` and `y[1]` first, then `x[2]` and `y[2]`, etc. `R` made things easy, by utilizing *vectorization*: it takes care of entire vectors at once, without explicit loops needing to be defined by you.

Question 3

Now redefine the vector `x` to be of length 500, with all the elements being 0. You don't want to do this using the `c()` function! (Look back at section 2.1 of [dmu-intro-r.github.io](#) for alternative ways to define vectors.)

```
x <- rep(c(0), each = 500)
length(x)

## [1] 500
```

Question 4

Let's define a random vector of integers with the `sample()` function. The information that you pass into a function is called an argument, and `R` functions sometimes can have many arguments. Let's look at the help page for `sample()`, which you can bring up by typing `?sample` in the console, or going to the `Help` pane and typing `sample` in the search bar.

```
Usage

sample(x, size, replace = FALSE, prob = NULL)
```

(What I just typed above is an example of a *verbatim* block: it **doesn't** execute as `R` code.) What we see is that `sample()` has four arguments. Two of them, `replace` and `prob`, have *default values*...so if you are happy with the defaults, you need not specify values for these arguments at all. You **do** need to specify any arguments which have no default value. Here, at a minimum, that would be two arguments: `x`, which is either a number or a vector from which to sample data, and `size`, which is the number of data to sample. If you do this

```
x <- sample(10, 5)
```

you are telling `R` to sample five numbers between 1 and 10 (inclusive), with all the numbers being different (because `replace = FALSE`, by default), and to save the numbers as the vector `x`. If you do this

```
x <- sample(40:50, 5)
```

you are telling `R` to sample five different numbers between 40 and 50 (inclusive, and without replacement). And if you do this

```
x <- sample(3, 10, replace = TRUE)
```

you are telling `R` to sample ten numbers between 1 and 3 (inclusive), and now with repetition being allowed. (We call this "sampling with replacement.") Etc. Now, sample 100 numbers between 1 and 100 (inclusive) with replacement, and save the output as the vector `x`. How many unique integers are there in `x`? Use handy vector functions (see section 2.3) to get a concise answer: do not print out `x` and count by eye! (If you need help, call a TA or me over, or come to office hours.)

```
x <- sample(1:100,size = 100,replace = TRUE)
unique(x)

## [1] 87 42 55 2 17 49 27 1 72 75 12 36 28 40 5 97 92 37 6 35 3 68 82 41 22
## [26] 65 19 99 93 32 53 45 39 98 31 38 63 14 46 52 56 7 44 57 71 80 83 60 4 86
## [51] 70 18 66 64 58 24 54 25 33 34
```

Question 5

Relational operators are binary operators of the form "`>`", e.g., `x < 0`. The six basic relational operators are `==`, `!=`, `<`, `>`, `<=`, and `>=` (for "equals," "not equals," "less than," "greater than," "less than or equals," and "greater than or equals"). Relational operators return a vector of logicals, meaning a vector of `TRUE` and `FALSE` values. Below, redefine `x` to be the vector with elements 1, 2, 1, and 8, and then display the output for `x == 1` and `x > 3`.

```
x <- c(1,2,1,8)
print(x==1)

## [1] TRUE FALSE TRUE FALSE

print(x > 3)

## [1] FALSE FALSE FALSE TRUE
```

Question 6

Apply the `sum()` function with input `x == 1`. Does the output make sense to you?

```
sum(x==1)

## [1] 2
```

Question 7

Relational operators may be combined with `&` (logical AND) or `|` (logical OR). Below, display the output for `x < 2 | x > 5`.

```
print(x<2 | x>5)

## [1] TRUE FALSE TRUE TRUE
```

Question 8

A reason to learn relational operators is that they underpin the manipulation of vectors (and thus underpin the manipulation of, e.g., rows or columns of data frames). To display a subset of values of the vector `x`, you can for instance type `x[...]`, where you would replace `...` with a relational operator. What happens when you type `x[x == 1]`?

```
print(x[x==1])

## [1] 1 1
```

Question 9

Some last things to do for now: apply the `length()` function to `x`, apply the `sort()` function to `x`, apply the `sort()` function to `x` with the additional argument `decreasing = TRUE`, apply the `unique()` function to `x`, and apply the `table()` function to `x`. (You may have done some similar things above when we told you to solve certain problems with handy vector functions.) Try to form some intuition about what each does. (Note: `table()` is a handy function for doing exploratory data analysis of categorical variables.)

```
length(x)

## [1] 4

sort(x, decreasing = TRUE)

## [1] 8 2 1 1

unique(x)

## [1] 1 2 8
```

Question 10

(Looking ahead.) A *list* in `R` is a collection of other objects, let's say of vectors, for now. Define a list below using `list()`, with the first argument being a defined vector with name `x` and values 1 and 2, and the second argument being a defined vector with name `y` and values "a", "b", and "c". (Note: your arguments won't look like `z <- c(TRUE, FALSE)` but instead like `z = c(TRUE, FALSE)`). Display the list.

```
x <- c(1,2)
y <-c("a", "b", "c")
z <-list(x = x, y = y)
print(z)

## $x
## [1] 1 2
##
## $y
## [1] "a" "b" "c"
```

The individual entries of a list are vectors, which are homogeneous, but the entries may each be of **different type**. A **list whose entries are all of the same length is a data frame (i.e., a structured data table)**.

Question 11

Download `simple.txt` from the Week 1 module page. Apply an appropriate function to read the file's contents into `R`. (Are the data separated by commas or spaces? Note that when you click on the file link, the file's contexts should appear in `Canvas`, along with a download link.) Show the names of the columns. Make sure the names are correct, and that there are eight columns. (Use `ncol()` to determine the number of columns.)

Note: you should consider including the argument `stringsAsFactors = FALSE`.

```
table <- read.table("C:/Users/Trish/Downloads/simple.txt", header = TRUE, stringsAsFactors = FALSE)
print(table)

##      name      u      g      r      i      z      y redshift
## 1 galaxy.A 17.8313 16.9077 16.4431 16.2099 16.0613 15.8732 0.038356
## 2 galaxy.B 19.0731 17.7448 16.9789 16.5288 16.2551 15.9531 0.058309
## 3 galaxy.C 21.6380 21.0106 20.8286 20.6283 20.6552 20.5280 0.063701
## 4 galaxy.D 20.5474 19.5542 19.2387 19.0568 19.0887 18.9865 0.059006
## 5 galaxy.E 21.2378 20.6876 20.5661 20.4371 20.4799 20.4503 0.063202
## 6 galaxy.F 22.4627 21.4597 21.0484 20.8274 20.7639 20.6385 0.057773
## 7 galaxy.G 23.8221 22.8950 22.5779 22.3543 22.3225 22.2038 0.061548
## 8 galaxy.H 23.0491 22.1536 21.8791 21.6889 21.7044 21.6381 0.063769
## 9 galaxy.I 23.6742 23.0346 22.7857 22.6116 22.5813 22.5462 0.061427
```

Question 12

Read in the data file from Question 11 but skip the header. Display the names that `R` gives to the columns. Note that the `skip` argument can be very useful for skipping over any metadata that may be present in your file.

```
table <- read.table("C:/Users/Trish/Downloads/simple.txt", header = FALSE, stringsAsFactors = FALSE, skip = 1)
print(table)

##      V1      V2      V3      V4      V5      V6      V7      V8
## 1 galaxy.A 17.8313 16.9077 16.4431 16.2099 16.0613 15.8732 0.038356
## 2 galaxy.B 19.0731 17.7448 16.9789 16.5288 16.2551 15.9531 0.058309
## 3 galaxy.C 21.6380 21.0106 20.8286 20.6283 20.6552 20.5280 0.063701
## 4 galaxy.D 20.5474 19.5542 19.2387 19.0568 19.0887 18.9865 0.059006
## 5 galaxy.E 21.2378 20.6876 20.5661 20.4371 20.4799 20.4503 0.063202
## 6 galaxy.F 22.4627 21.4597 21.0484 20.8274 20.7639 20.6385 0.057773
## 7 galaxy.G 23.8221 22.8950 22.5779 22.3543 22.3225 22.2038 0.061548
## 8 galaxy.H 23.0491 22.1536 21.8791 21.6889 21.7044 21.6381 0.063769
## 9 galaxy.I 23.6742 23.0346 22.7857 22.6116 22.5813 22.5462 0.061427
```

Question 13

Read in the data file from Question 11 but only read in the first four lines, while retaining the header. Print the data frame. (Note: in general, you can use `skip` and `nrows` to zero in on portions of a text file where the data actually reside.)

```
table <- read.table("C:/Users/Trish/Downloads/simple.txt", header = TRUE, stringsAsFactors = FALSE, nrows = 4)
print(table)

##      name      u      g      r      i      z      y redshift
## 1 galaxy.A 17.8313 16.9077 16.4431 16.2099 16.0613 15.8732 0.038356
## 2 galaxy.B 19.0731 17.7448 16.9789 16.5288 16.2551 15.9531 0.058309
## 3 galaxy.C 21.6380 21.0106 20.8286 20.6283 20.6552 20.5280 0.063701
## 4 galaxy.D 20.5474 19.5542 19.2387 19.0568 19.0887 18.9865 0.059006
```

Question 14

Download `planets_small.csv` from the Week 1 module page. Apply an appropriate function to read the file's contents into `R`. Note that here, you have one column that should be rendered as character strings (`pl_hostname`, the first column), while the rest should be rendered as factor variables. Thus you do not want to use the `stringsAsFactors` argument here, as it is too coarse. You need to explicitly specify the types of each column, using the functional argument `colClasses`. After reading the data in, pass your data frame into the function `summary()` and see what happens.

```
csv <-read.csv("C:/Users/Trish/Downloads/planets_small.csv", header = TRUE, colClasses =c('character', 'factor', 'factor', 'factor'),nrows = 5)
print(csv)

##      pl_hostname pl_letter      pl_discmethod pl_pnum
## 1      11 Com          b Radial Velocity          1
## 2      11 UMi          b Radial Velocity          1
## 3      14 And          b Radial Velocity          1
## 4      14 Her          b Radial Velocity          1
## 5      16 Cyg B          b Radial Velocity          1
```

Question 15

Download `emline.csv` from the Week 1 module page. Apply an appropriate function to read the file's contents into `R`. When you are done, show the mean and median values of the `sfr` column. *Hint*: if they are wildly different, you may need to adjust how you read in the data. *Hint*: look for numbers that represent missing data, and use an appropriate argument to tell `R` that those numbers should be converted to `NA`. *Note*: if any are converted to `NA`, you will have to pass an additional argument into the `mean()` and `median()` functions: `na.rm = TRUE`, which tells `R` to ignore the `NA`s in computations.

```
df <-read.csv("C:/Users/Trish/Downloads/emline.csv", header = TRUE,na.strings = "NA")
print(df)

##      O_II_3729 H_gamma      H_beta O_III_4959 O_III_5007 N_II_6548 H_ALPHA N_II_6584
## 1      1.5364      1.7004      0.3709      -0.0549      0.6216      0.4846      1.2458      0.5687
## 2      9.4411      2.7609      4.0603      0.5418      0.8290      0.7808      11.2330      4.2059
## 3      17.3146      3.3528      6.3293      0.7573      4.7597      1.0888      21.6571      3.5426
## 4      8.8070      -0.2765      1.4274      1.0690      1.7463      0.5824      7.5148      2.0032
## 5      17.3470      3.8775      2.6622      0.5714      3.2273      1.1112      13.9770      4.1352
## 6      37.8111      4.4961      10.0149      3.2715      11.7268      2.2371      39.9036      8.3394
## 7      55.3728      7.7921      13.7503      9.4787      28.0838      1.3237      38.2856      3.9767
## 8      24.0244      4.6931      8.4271      1.9320      6.4095      2.6659      35.1531      10.7334
## 9      31.8309      3.2743      7.0494      3.7526      12.1558      2.3935      31.8069      5.3516
##      S_II_6717 S_II_6731      mass      sfr
## 1      -0.4526      1.0754      9.9484      -9999.0000
## 2      1.8614      2.6442      9.9976      -9999.0000
## 3      2.5171      3.1083      9.8058      -0.7249
## 4      2.7998      0.7398      9.8352      -9999.0000
## 5      4.6136      2.3890      9.7140      -0.6436
## 6      9.9219      6.2621      9.6657      -0.4929
## 7      3.2716      4.6510      8.9972      -9999.0000
## 8      9.7281      5.5440      9.8578      -0.5099
## 9      8.5123      10.3885      9.8120      -1.1993

df$sfr[df$sfr == -9999] <-NA
print(median(df$sfr, na.rm = TRUE))

## [1] -0.71412

print(median(df$sfr, na.rm = TRUE))

## [1] -0.6436
```